Ich habe Flgelsignale von Viessman, z.B. 4500 deren Antrieb nicht mehr langsam läuft. Ich möchte diese nun umbauen und mit einem Servo ansteuern. Ich habe einen Arduino und einen Servotreiber https://a.co/d/0b1xNKcV für die Ansteuerung. Meine Zentrale ist die originale Intellibox I.

Torsten, dein Ansatz ist absolut sinnvoll – die alten Viessmann-Flügelsignale (z. B. 4500) leiden oft darunter, dass der Spulenantrieb ruckartig oder gar nicht mehr sauber läuft. Der Umbau auf Servoantrieb ist eine bewährte Lösung mit deutlich besserer Optik und Kontrolle.

Ich teile das in drei Bereiche: **Mechanik, Elektrik, Ansteuerung mit Intellibox**.

---

1. Mechanischer Umbau

Ziel: Spulenantrieb raus → Servo übernimmt Bewegung

Vorgehen

- Originalantrieb komplett entfernen
- Flügelmechanik beibehalten (wichtig!)
- Servo unter der Anlage montieren
- Verbindung:
  - dünner Draht (0,8–1 mm Federstahldraht)
  - durch die Stellstange führen

Tipps

- Kleine Servos wie **SG90 oder MG90S** reichen völlig
- Endlagen mechanisch nicht erzwingen → lieber per Software begrenzen
- Langsame Bewegung = später per Arduino

---

2. Elektrik (Arduino + Servotreiber)

Dein verlinkter Treiber ist sehr wahrscheinlich ein **PCA9685 (16-Kanal Servo-Driver)**.

Verkabelung

PCA9685 → Arduino

- VCC → 5V
- GND → GND
- SDA → A4 (Uno)
- SCL → A5 (Uno)

Servo

- Rot → externe 5V (WICHTIG, nicht vom Arduino!)
- Braun/Schwarz → GND
- Gelb/Orange → PWM vom PCA9685

👉 Unbedingt:

- Gemeinsame Masse zwischen Arduino + Servonetzteil

---

3. Ansteuerung über Intellibox I

Hier liegt der spannendste Teil.

Die Intellibox kann:

- DCC Zubehörbefehle senden
- KEIN direktes Servo-Signal

Lösung: Arduino als DCC-Decoder

Du brauchst:
👉 **DCC Signal einlesen → Servo bewegen**

Optionen:

Option A (empfohlen)
Nutze Library:

- **NmraDcc**

Damit wird dein Arduino ein Zubehördecoder.

---

4. Beispiel: DCC → Servo

Grundidee:

- Intellibox sendet „Signal rot/grün“
- Arduino empfängt Adresse
- Servo fährt langsam in Position

Beispielcode (gekürzt, Kernlogik)

```
#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>
#include <NmraDcc.h>

NmraDcc Dcc;
Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver();

#define SERVO_MIN 150
#define SERVO_MAX 500

int pos = 300;
int target = 300;

void notifyDccAccTurnoutOutput(uint16_t Addr, uint8_t Direction, uint8_t OutputPower)
{
  if (Addr == 1) {
    if (Direction == 0) target = 200; // Hp0
    else target = 400;                // Hp1
  }
}

void setup() {
  pwm.begin();
  pwm.setPWMFreq(50);

  Dcc.pin(2, 0); // DCC Signal an Pin 2
  Dcc.init(MAN_ID_DIY, 10, 0, 0);
}

void loop() {
  Dcc.process();

  if (pos < target) pos++;
```

```
  if (pos > target) pos--;

  pwm.setPWM(0, 0, pos);
```

C++

👉 Ergebnis:

- weiche Bewegung
- realistische Signalstellung

---

5. Anschluss an die Intellibox

DCC Signal abgreifen

- Gleisausgang (über Optokoppler empfohlen)
- oder:
  - fertiges DCC-Interface-Modul nutzen

👉 Direktanschluss ohne Schutz kann den Arduino zerstören

---

6. Alternative (komfortabler)

Wenn du weniger basteln willst:

Fertige Lösungen:

- ESU SwitchPilot Servo
- MBTronik ServoDecoder
- Qdecoder

👉 Vorteil:

- direkt kompatibel mit Intellibox
- kein Programmieren nötig

👉 Nachteil:

- weniger flexibel
- teurer

---

7. Feinabstimmung (entscheidend für Realismus)

- Geschwindigkeit im Code einstellen
- Endlagen exakt kalibrieren
- kleine Bewegungsspielräume lassen
- ggf.:
  - leichtes „Nachfedern“ simulieren

Fazit

Dein Setup ist technisch absolut solide:

- Arduino + PCA9685 = perfekt für viele Signale
- Intellibox liefert DCC → gut integrierbar
- Servo = beste Lösung für realistische Bewegung

Wenn du möchtest, kann ich dir als nächsten Schritt:

- eine **komplette Schaltung inkl. Optokoppler** zeichnen
- oder den Code für **mehrere Signale + Adressen** erweitern
- oder dir eine **plug-and-play Architektur für 10–20 Signale** bauen

Ich habe eine Interface Platine für DCC oder Loconet. Nachfedern wäre toll.

Thought for a couple of seconds

Torsten, perfekt — mit einer **DCC- oder LocoNet-Interface-Platine** wird das sauberer als direkt am Gleis.

Für dein Ziel würde ich so aufbauen:

```
Intellibox I
   ↓ DCC oder LocoNet
Interface-Platine
   ↓ Arduino
PCA9685 Servotreiber
```

```
   ↓
Servo am Viessmann-Flügelsignal
```

Empfehlung

Wenn deine Interface-Platine LocoNet sauber unterstützt:
Nimm **LocoNet**. Die Intellibox I spricht LocoNet nativ, und Zubehörbefehle lassen sich sehr gut auswerten.

Wenn die Platine eher DCC-Zubehördecoder-Signal liefert:
Dann DCC mit `NmraDcc` nehmen.

Nachfedern / Wippen des Flügels

Die Servo-Bewegung sollte nicht einfach linear laufen, sondern so:

1. langsam zur Zielposition fahren
2. minimal über das Ziel hinaus
3. zurückfedern
4. kleiner Gegenimpuls
5. endgültig einpendeln

Beispiel:

```cpp
void moveSignalWithBounce(uint8_t channel, int from, int to) {
  int overshoot = (to > from) ? 18 : -18;
  int rebound   = (to > from) ? -8 : 8;

  slowMove(channel, from, to + overshoot, 12);
  slowMove(channel, to + overshoot, to + rebound, 18);
```

‹/› C++

Dazu die Grundfunktion:

```cpp
void slowMove(uint8_t channel, int from, int to, int delayMs) {
  if (from < to) {
    for (int p = from; p <= to; p++) {
      pwm.setPWM(channel, 0, p);
      delay(delayMs);
```

```
    }
  } else {
    for (int p = from; p >= to; p--) {
      pwm.setPWM(channel, 0, p);
      delay(delayMs);
    }
```

C++

Beispielwerte für ein Viessmann-Flügelsignal

```
int hp0 = 260;   // Halt
```

C++

Beim Schalten:

```
if (signalSollAufFahrt) {
  moveSignalWithBounce(0, currentPos, hp1);
  currentPos = hp1;
}

if (signalSollAufHalt) {
  moveSignalWithBounce(0, currentPos, hp0);
```

C++

Realistische Feineinstellung

Typische gute Werte:

```
overshoot: 10–25
```

C++

Für Flügelsignale wirkt meist gut:

C++

Wichtig

Servo-Endlagen nie mechanisch an den Anschlag fahren lassen. Lieber im Code begrenzen und den Stelldraht so einstellen, dass noch etwas Spiel bleibt.

Für mehrere Signale würde ich eine Tabelle mit Adresse, Servo-Kanal, Hp0-Position, Hp1-Position und aktueller Position verwenden.

Ich habe insgesamt 5 zweibegriffige Flügelsignale, 1 Sperrsignal, 1 zweibegriffiges und 1 dreibegriffiges Vorsignal.

Thought for a couple of seconds

Torsten, das passt sehr gut auf **einen PCA9685 mit 16 Servo-Kanälen**.

Benötigte Servo-Kanäle

Signaltyp	Anzahl	Servos je Signal	Kanäle
Zweibegriffige Hauptsignale	5	1	5
Sperrsignal	1	1	1
Zweibegriffiges Vorsignal	1	1	1
Dreibegriffiges Vorsignal	1	meist 2	2
Summe	8 Signale		**9 Kanäle**

Du hast also noch **7 freie Servo-Kanäle**.

Adressvorschlag Intellibox

DCC/LocoNet-Adresse	Signal	Servo-Kanal
1	Hauptsignal 1	0
2	Hauptsignal 2	1
3	Hauptsignal 3	2
4	Hauptsignal 4	3
5	Hauptsignal 5	4
6	Sperrsignal	5
7	Vorsignal 2-begriffig	6
8	Vorsignal 3-begriffig, Flügel/Scheibe 1	7
9	Vorsignal 3-begriffig, Flügel/Scheibe 2	8

Zustände

Hauptsignal zweibegriffig

```
Hp0 = Halt
Hp1 = Fahrt
```

Sperrsignal

```
Sh0 = Halt
Sh1 = Fahrverbot aufgehoben
```

Vorsignal zweibegriffig

```
Vr0 = Halt erwarten
Vr1 = Fahrt erwarten
```

Vorsignal dreibegriffig

```
Vr0 = Halt erwarten
Vr1 = Fahrt erwarten
Vr2 = Langsamfahrt erwarten
```

Beim dreibegriffigen Vorsignal brauchst du vermutlich **zwei Servos**, weil zwei bewegliche Elemente getrennt gestellt werden müssen.

Datenstruktur im Arduino

So würde ich es aufbauen:

```cpp
struct ServoSignal {
  uint16_t address;
  uint8_t channel;
  int pos0;
  int pos1;
  int currentPos;
};

ServoSignal signals[] = {
  {1, 0, 260, 410, 260}, // Hauptsignal 1
  {2, 1, 260, 410, 260}, // Hauptsignal 2
  {3, 2, 260, 410, 260},
  {4, 3, 260, 410, 260},
  {5, 4, 260, 410, 260},
  {6, 5, 270, 390, 270}, // Sperrsignal
  {7, 6, 250, 405, 250}, // Vorsignal 2-begriffig

  // Dreibegriffiges Vorsignal
  {8, 7, 250, 405, 250},
```

C++

Nachfedern je Signaltyp

Für Haupt- und Vorsignale:

```cpp
```

C++

Für Sperrsignal eher kleiner:

C++

Nächster sinnvoller Aufbau

Ich würde es so machen:

1. Erst **ein Hauptsignal** mechanisch umbauen.
2. Endlagen `Hp0` und `Hp1` sauber kalibrieren.
3. Nachfedern einstellen.
4. Danach die Werte auf die anderen Signale übertragen.
5. Dreibegriffiges Vorsignal zuletzt, weil es die komplexeste Logik hat.

Für deine Anlage reicht also: **1 Arduino + 1 PCA9685 + 5V-Servonetzteil + deine DCC/LocoNet-Platine**.

Das wippen ist häufig ein paar Mal federn bis die Ruhelage erreicht wurde.

Genau, Torsten — dann sollte das Nachfedern als **gedämpfte Schwingung** laufen, nicht nur als einmaliges Überschwingen.

Prinzip:

```
Start → Ziel überschießen → zurück → kleiner überschießen → zurück → noch kleiner → Ru
```

Beispiel-Funktion:

```cpp
void moveSignalWithDampedBounce(uint8_t channel, int from, int to) {
  int dir = (to > from) ? 1 : -1;

  slowMove(channel, from, to + dir * 18, 12);
  slowMove(channel, to + dir * 18, to - dir * 10, 18);
  slowMove(channel, to - dir * 10, to + dir * 6, 22);
```

```
  slowMove(channel, to + dir * 6,  to - dir * 3, 26);
```

C++

Mit Grundfunktion:

```
void slowMove(uint8_t channel, int from, int to, int delayMs) {
  int step = (to > from) ? 1 : -1;

  for (int p = from; p != to; p += step) {
    pwm.setPWM(channel, 0, p);
    delay(delayMs);
  }
```

C++

Für Flügelsignale wirken diese Werte gut:

C++

Für stärkeres Wippen:

C++

Für dezenteres Wippen:

C++

Die schönste Variante wäre parametrierbar:

```
int bounce[] = {18, -10, 6, -3, 0};

void moveWithBouncePattern(uint8_t channel, int from, int to, int *pattern, int count) {
  int dir = (to > from) ? 1 : -1;
  int current = from;
```

```
  for (int i = 0; i < count; i++) {
    int next = to + dir * pattern[i];
    slowMove(channel, current, next, 18 + i * 4);
    current = next;
```

C++

Aufruf:

```
int mainSignalBounce[] = {18, -10, 6, -3, 0};
```

C++

Damit kannst du später pro Signaltyp eigene Bewegungen definieren:


```
int mainSignalBounce[] = {18, -10, 6, -3, 0};
```

C++

Das sieht deutlich realistischer aus als ein Servo, das nur langsam von A nach B fährt.

https://youtube.com/shorts/Gdxv3Rbquto?si=Aeu5o3s6ow2qUUNo

Hier ist ein Beispiel

Ich schaue mir an, ob die Videoseite genug verwertbare Informationen liefert. Danach übersetze ich die sichtbare Bewegung in ein Servo-Bewegungsprofil mit mehreren gedämpften Ausschlägen.

Thought for a couple of seconds

Torsten, der YouTube-Link lässt sich hier nicht zuverlässig abrufen. Für das typische Flügelsignal-Wippen würde ich das Servo-Profil so modellieren:

```cpp
int mainSignalBounce[] = {
   22,   // erstes Überschwingen
  -15,   // kräftiges Zurückfedern
   10,
   -7,
    4,
   -2,
    1,
```

C++

Mit langsamer werdender Bewegung:

```cpp
void moveWithSignalBounce(uint8_t channel, int from, int to) {
  int bounce[] = {22, -15, 10, -7, 4, -2, 1, 0};
  int delayMs[] = {10, 13, 16, 20, 25, 31, 38, 45};

  int dir = (to > from) ? 1 : -1;
  int current = from;

  for (int i = 0; i < 8; i++) {
    int next = to + dir * bounce[i];
    slowMove(channel, current, next, delayMs[i]);
    current = next;
```

C++

Das ergibt: **zügiges Stellen → deutliches mehrfaches Federn → langsames Einpendeln**.

Für sehr altes/leichtes Signal:

C++

Für dezenter:

C++

Auch beim Rotstellen wippt es.

Ja, Torsten — dann sollte **jede Stellrichtung** wippen: Hp0 → Hp1 **und** Hp1 → Hp0.

Der Code macht das bereits, wenn `dir` aus `from` und `to` berechnet wird:

```cpp
void moveWithSignalBounce(uint8_t channel, int from, int to) {
  int bounce[] = {22, -15, 10, -7, 4, -2, 1, 0};
  int delayMs[] = {10, 13, 16, 20, 25, 31, 38, 45};

  int dir = (to > from) ? 1 : -1;
  int current = from;

  for (int i = 0; i < 8; i++) {
    int next = to + dir * bounce[i];
    slowMove(channel, current, next, delayMs[i]);
    current = next;
```

C++

Beim Rotstellen passiert dadurch automatisch:

```
Hp1 → Hp0 überschießen → zurückfedern → kleiner überschießen → Ruhelage Hp0
```

Aufruf:

```cpp
if (signalAufRot) {
  moveWithSignalBounce(0, currentPos, hp0);
  currentPos = hp0;
}

if (signalAufGruen) {
  moveWithSignalBounce(0, currentPos, hp1);
```

C++

```
}
```

Wichtig: Die Ausschläge dürfen beim Rotstellen nicht gegen den mechanischen Anschlag drücken. Deshalb `hp0` etwas vom echten Anschlag weg kalibrieren.

Bitte komplettes Program erstellen

Ich baue dir ein vollständiges Arduino-Programm mit Tabellenkonfiguration für alle 9 Servokanäle, gedämpftem Wippen in beide Richtungen und DCC-Zubehöradressen als Standardannahme.

Thought for 7s

Torsten, hier ist ein vollständiges Programm für:

- 5 zweibegriffige Hauptsignale
- 1 Sperrsignal
- 1 zweibegriffiges Vorsignal
- 1 dreibegriffiges Vorsignal mit 2 Servos
- PCA9685 Servotreiber
- DCC über Interface-Platine
- Wippen in beide Richtungen

```
#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>
#include <NmraDcc.h>

Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver();
NmraDcc Dcc;

// DCC-Eingang vom Interface
#define DCC_PIN 2

// Servo-Kanäle PCA9685
#define HS1 0
#define HS2 1
```

```
#define HS3 2
#define HS4 3
#define HS5 4
#define SPERR 5
#define VS2 6
#define VS3_A 7
#define VS3_B 8

// Servo-Positionen: bitte später kalibrieren
int posHp0 = 260;
int posHp1 = 410;

int posSh0 = 270;
int posSh1 = 390;

int posVr0 = 250;
int posVr1 = 405;
int posVr2a = 330;
int posVr2b = 405;

// Aktuelle Positionen
int currentPos[16] = {
  posHp0, posHp0, posHp0, posHp0, posHp0,
  posSh0,
  posVr0,
  posVr0, posVr0
};

void setup() {
  Wire.begin();

  pwm.begin();
  pwm.setPWMFreq(50);

  delay(500);

  Dcc.pin(DCC_PIN, 0);
  Dcc.init(MAN_ID_DIY, 10, 0, 0);

  // Startpositionen setzen
  setServo(HS1, posHp0);
  setServo(HS2, posHp0);
  setServo(HS3, posHp0);
  setServo(HS4, posHp0);
  setServo(HS5, posHp0);
  setServo(SPERR, posSh0);
  setServo(VS2, posVr0);
  setServo(VS3_A, posVr0);
  setServo(VS3_B, posVr0);
```

```
}

void loop() {
  Dcc.process();
}

// DCC-Zubehörbefehl von Intellibox
void notifyDccAccTurnoutOutput(uint16_t Addr, uint8_t Direction, uint8_t OutputPower) {
  if (!OutputPower) return;

  switch (Addr) {

    // Hauptsignale 1–5
    case 1:
      moveSignal(HS1, Direction ? posHp1 : posHp0);
      break;

    case 2:
      moveSignal(HS2, Direction ? posHp1 : posHp0);
      break;

    case 3:
      moveSignal(HS3, Direction ? posHp1 : posHp0);
      break;

    case 4:
      moveSignal(HS4, Direction ? posHp1 : posHp0);
      break;

    case 5:
      moveSignal(HS5, Direction ? posHp1 : posHp0);
      break;

    // Sperrsignal
    case 6:
      moveSignal(SPERR, Direction ? posSh1 : posSh0);
      break;

    // Zweibegriffiges Vorsignal
    case 7:
      moveSignal(VS2, Direction ? posVr1 : posVr0);
      break;

    // Dreibegriffiges Vorsignal:
    // Adresse 8: Vr0 / Vr1
    case 8:
      if (Direction) {
        moveSignal(VS3_A, posVr1);
        moveSignal(VS3_B, posVr0);
```

```
      } else {
        moveSignal(VS3_A, posVr0);
        moveSignal(VS3_B, posVr0);
      }
      break;

    // Adresse 9: Vr2 / Vr0
    case 9:
      if (Direction) {
        moveSignal(VS3_A, posVr2a);
        moveSignal(VS3_B, posVr2b);
      } else {
        moveSignal(VS3_A, posVr0);
        moveSignal(VS3_B, posVr0);
      }
      break;
  }
}

void moveSignal(uint8_t channel, int target) {
  int from = currentPos[channel];

  if (from == target) return;

  moveWithBounce(channel, from, target);
  currentPos[channel] = target;
}

void moveWithBounce(uint8_t channel, int from, int to) {
  int bounce[]  = {22, -15, 10, -7, 4, -2, 1, 0};
  int delays[]  = {10, 13, 16, 20, 25, 31, 38, 45};

  int dir = (to > from) ? 1 : -1;
  int current = from;

  for (int i = 0; i < 8; i++) {
    int next = to + dir * bounce[i];

    // Sicherheitsbegrenzung
    next = constrain(next, 120, 560);

    slowMove(channel, current, next, delays[i]);
    current = next;
  }
}

void slowMove(uint8_t channel, int from, int to, int delayMs) {
  int step = (to > from) ? 1 : -1;
```

```
  for (int p = from; p != to; p += step) {
    setServo(channel, p);
    delay(delayMs);
  }

  setServo(channel, to);
}

void setServo(uint8_t channel, int pulse) {
```

C++

Wichtigste Stellen zum Anpassen:

C++

Und für stärkeres Wippen:

C++

Für dezenteres Wippen:

C++

Für deine Intellibox verwendest du dann Zubehöradressen **1 bis 9**.

Bitte auch Alternative für Loconet shield anstelle von DCC erstellen

Ich erstelle dir die LocoNet-Variante mit derselben Servo-Logik wie im DCC-Programm: gleiche Adressen, gleiche Kanäle, gleiches Wippen, nur anderer Eingang über LocoNet.

Thought for a few seconds

Torsten, hier ist die **LocoNet-Shield-Alternative**. Sie ersetzt nur den DCC-Teil; die Servo-Logik bleibt gleich.

Benötigte Library in Arduino IDE:

```
LocoNet
Adafruit PWM Servo Driver
```

```
#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>
#include <LocoNet.h>

Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver();

// LocoNet Shield RX/TX Pin je nach Shield prüfen
#define LOCONET_TX_PIN 7

// PCA9685 Kanäle
#define HS1    0
#define HS2    1
#define HS3    2
#define HS4    3
#define HS5    4
#define SPERR  5
#define VS2    6
#define VS3_A  7
#define VS3_B  8

// Servo-Positionen kalibrieren
int posHp0 = 260;
int posHp1 = 410;

int posSh0 = 270;
int posSh1 = 390;

int posVr0  = 250;
int posVr1  = 405;
int posVr2a = 330;
int posVr2b = 405;

int currentPos[16] = {
  posHp0, posHp0, posHp0, posHp0, posHp0,
  posSh0,
```

```
  posVr0,
  posVr0, posVr0
};

void setup() {
  Wire.begin();

  pwm.begin();
  pwm.setPWMFreq(50);
  delay(500);

  LocoNet.init(LOCONET_TX_PIN);

  setServo(HS1, posHp0);
  setServo(HS2, posHp0);
  setServo(HS3, posHp0);
  setServo(HS4, posHp0);
  setServo(HS5, posHp0);
  setServo(SPERR, posSh0);
  setServo(VS2, posVr0);
  setServo(VS3_A, posVr0);
  setServo(VS3_B, posVr0);
}

void loop() {
  lnMsg *msg = LocoNet.receive();

  if (msg) {
    LocoNet.processSwitchSensorMessage(msg);
  }
}

// Wird von der LocoNet-Library bei Weichen-/Zubehörbefehlen aufgerufen
void notifySwitchRequest(uint16_t address, uint8_t output, uint8_t direction) {
  if (!output) return;

  bool thrown = direction;

  switch (address) {

    case 1:
      moveSignal(HS1, thrown ? posHp1 : posHp0);
      break;

    case 2:
      moveSignal(HS2, thrown ? posHp1 : posHp0);
      break;

    case 3:
```

```
      moveSignal(HS3, thrown ? posHp1 : posHp0);
      break;

    case 4:
      moveSignal(HS4, thrown ? posHp1 : posHp0);
      break;

    case 5:
      moveSignal(HS5, thrown ? posHp1 : posHp0);
      break;

    case 6:
      moveSignal(SPERR, thrown ? posSh1 : posSh0);
      break;

    case 7:
      moveSignal(VS2, thrown ? posVr1 : posVr0);
      break;

    // Dreibegriffiges Vorsignal: Vr0 / Vr1
    case 8:
      if (thrown) {
        moveSignal(VS3_A, posVr1);
        moveSignal(VS3_B, posVr0);
      } else {
        moveSignal(VS3_A, posVr0);
        moveSignal(VS3_B, posVr0);
      }
      break;

    // Dreibegriffiges Vorsignal: Vr2 / Vr0
    case 9:
      if (thrown) {
        moveSignal(VS3_A, posVr2a);
        moveSignal(VS3_B, posVr2b);
      } else {
        moveSignal(VS3_A, posVr0);
        moveSignal(VS3_B, posVr0);
      }
      break;
  }
}

void moveSignal(uint8_t channel, int target) {
  int from = currentPos[channel];

  if (from == target) return;

  moveWithBounce(channel, from, target);
```

```
  currentPos[channel] = target;
}

void moveWithBounce(uint8_t channel, int from, int to) {
  int bounce[] = {22, -15, 10, -7, 4, -2, 1, 0};
  int delays[] = {10, 13, 16, 20, 25, 31, 38, 45};

  int dir = (to > from) ? 1 : -1;
  int current = from;

  for (int i = 0; i < 8; i++) {
    int next = to + dir * bounce[i];
    next = constrain(next, 120, 560);

    slowMove(channel, current, next, delays[i]);
    current = next;
  }
}

void slowMove(uint8_t channel, int from, int to, int delayMs) {
  int step = (to > from) ? 1 : -1;

  for (int p = from; p != to; p += step) {
    setServo(channel, p);
    delay(delayMs);
  }

  setServo(channel, to);
}

void setServo(uint8_t channel, int pulse) {
```

‹/› C++

Wichtig: Beim LocoNet Shield ggf. diesen Pin anpassen:

‹/› C++

Die Zubehöradressen bleiben:

```
1–5  Hauptsignale
6    Sperrsignal
7    zweibegriffiges Vorsignal
8–9  dreibegriffiges Vorsignal
```

Gut und nun ein erstes Shield ohne DCC oder Loconet zum testen am PC (USB). Vielleicht über Tastatur oder serielle Schnittstelle ansteuern.

Thought for a couple of seconds

Ja, Torsten — gute Idee. Für den ersten Test brauchst du nur:

```
PC USB → Arduino → PCA9685 → Servos
```

Keine Intellibox, kein DCC, kein LocoNet.

Steuerung über seriellen Monitor

Befehle:

```
1r  Signal 1 auf Halt
1g  Signal 1 auf Fahrt
2r
2g
...
6r  Sperrsignal Sh0
6g  Sperrsignal Sh1
7r  Vorsignal Vr0
7g  Vorsignal Vr1
8r  dreibegriffiges Vorsignal Vr0
8g  dreibegriffiges Vorsignal Vr1
8y  dreibegriffiges Vorsignal Vr2
a0  alle auf Halt/Vr0
```

Komplettes Testprogramm USB/Serial

```
#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>

Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver();
```

```
#define HS1    0
#define HS2    1
#define HS3    2
#define HS4    3
#define HS5    4
#define SPERR  5
#define VS2    6
#define VS3_A  7
#define VS3_B  8

int posHp0 = 260;
int posHp1 = 410;

int posSh0 = 270;
int posSh1 = 390;

int posVr0  = 250;
int posVr1  = 405;
int posVr2a = 330;
int posVr2b = 405;

int currentPos[16];

void setup() {
  Serial.begin(115200);
  Wire.begin();

  pwm.begin();
  pwm.setPWMFreq(50);

  delay(500);

  initPositions();

  Serial.println("Servo-Signaltester bereit.");
  Serial.println("Befehle: 1r, 1g ... 8r, 8g, 8y, a0");
}

void loop() {
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n');
    cmd.trim();
    handleCommand(cmd);
  }
}

void initPositions() {
  for (int i = 0; i < 16; i++) currentPos[i] = 300;
```

```
  setAndRemember(HS1, posHp0);
  setAndRemember(HS2, posHp0);
  setAndRemember(HS3, posHp0);
  setAndRemember(HS4, posHp0);
  setAndRemember(HS5, posHp0);
  setAndRemember(SPERR, posSh0);
  setAndRemember(VS2, posVr0);
  setAndRemember(VS3_A, posVr0);
  setAndRemember(VS3_B, posVr0);
}

void handleCommand(String cmd) {
  if (cmd.length() < 2) return;

  char id = cmd.charAt(0);
  char state = cmd.charAt(1);

  if (id == 'a' && state == '0') {
    allStop();
    return;
  }

  switch (id) {
    case '1': moveSignal(HS1, state == 'g' ? posHp1 : posHp0); break;
    case '2': moveSignal(HS2, state == 'g' ? posHp1 : posHp0); break;
    case '3': moveSignal(HS3, state == 'g' ? posHp1 : posHp0); break;
    case '4': moveSignal(HS4, state == 'g' ? posHp1 : posHp0); break;
    case '5': moveSignal(HS5, state == 'g' ? posHp1 : posHp0); break;

    case '6': moveSignal(SPERR, state == 'g' ? posSh1 : posSh0); break;

    case '7': moveSignal(VS2, state == 'g' ? posVr1 : posVr0); break;

    case '8':
      if (state == 'r') {
        moveSignal(VS3_A, posVr0);
        moveSignal(VS3_B, posVr0);
      } else if (state == 'g') {
        moveSignal(VS3_A, posVr1);
        moveSignal(VS3_B, posVr0);
      } else if (state == 'y') {
        moveSignal(VS3_A, posVr2a);
        moveSignal(VS3_B, posVr2b);
      }
      break;
  }

  Serial.print("OK: ");
  Serial.println(cmd);
```

```
}

void allStop() {
  moveSignal(HS1, posHp0);
  moveSignal(HS2, posHp0);
  moveSignal(HS3, posHp0);
  moveSignal(HS4, posHp0);
  moveSignal(HS5, posHp0);
  moveSignal(SPERR, posSh0);
  moveSignal(VS2, posVr0);
  moveSignal(VS3_A, posVr0);
  moveSignal(VS3_B, posVr0);

  Serial.println("Alle Signale auf Halt/Vr0.");
}

void moveSignal(uint8_t channel, int target) {
```






```
  moveWithBounce(channel, from, target);
  currentPos[channel] = target;
}

void moveWithBounce(uint8_t channel, int from, int to) {
  int bounce[] = {22, -15, 10, -7, 4, -2, 1, 0};
  int delays[] = {10, 13, 16, 20, 25, 31, 38, 45};

  int dir = (to > from) ? 1 : -1;
  int current = from;

  for (int i = 0; i < 8; i++) {
    int next = to + dir * bounce[i];
    next = constrain(next, 120, 560);
    slowMove(channel, current, next, delays[i]);
    current = next;
  }
}

void slowMove(uint8_t channel, int from, int to, int delayMs) {
  int step = (to > from) ? 1 : -1;

  for (int p = from; p != to; p += step) {
    setServo(channel, p);
    delay(delayMs);
  }

  setServo(channel, to);
```

C++



```
void setServo(uint8_t channel, int pulse) {
  pwm.setPWM(channel, 0, pulse);
}

void setAndRemember(uint8_t channel, int pulse) {
  setServo(channel, pulse);
  currentPos[channel] = pulse;
}
```

Serieller Monitor:

```
115200 Baud
Zeilenende: Newline
```

Zum Kalibrieren später am besten noch Befehle wie `p0 260` oder `s1 300` ergänzen.